# Python Input

To read from the keyboard (standard input), use the method `input`:

```
name = input("What's your name?")
print(name)
age = input("What's your age?")
print(age)
```

Prompt for value from user

print out value received

- Note in Python 2 the method is called `raw_input()`.

# Try it: Python Input, Output, and Dates

***Question 1:*** Write a program that reads a name and prints out the name, the length of the name, the first five characters of the name.

***Question 2:*** Print out the current date in `YYYY/MM/DD` format.

# Comparisons

A ***comparison operator*** compares two values. Examples:

- `5 < 10`
- `N > 5  # N is a variable. Answer depends on what is N.`

Comparison operators in Python:

- `>` - Greater than
- `>=` - Greater than or equal
- `<` - Less than
- `<=` - Less than or equal
- `==` - Equal (Note: Not "=" which is used for assignment!)
- `!=` - Not equal

The result of a comparison is a ***Boolean value*** which is either **`True`** or **`False`**.

# Conditions with `and`, `or`, `not`

A ***condition*** is an expression that is either `True` or `False` and may contain one or more comparisons. Conditions may be combined using: `and`, `or`, `not`.

- order of evaluation: `not`, `and`, `or` May change order with parentheses.

| Operation | Syntax | Examples | Output |
|---|---|---|---|
| **AND**<br>**(True if both are True)** | `and` | `True and True`<br>`False and True`<br>`False and False` | `True`<br>`False`<br>`False` |
| **OR**<br>**(True if either or both are True)** | `or` | `True or True`<br>`False or True`<br>`False or False` | `True`<br>`True`<br>`False` |
| **NOT**<br>**(Reverses: e.g. True becomes False)** | `not` | `not True`<br>`not False` | `False`<br>`True` |

# Condition Examples

```
n = 5
v = 8
print(n > 5)                          # False
print(n == v)                         # False
print(n != v)                         # True
print(n == v and n+4>v)               # False
print(n == v or n+4>v)                # True
print(n+1 == v-2 or not v>4)          # True
```

# Python Condition Question

***Question:*** How many of the following conditions are **TRUE**?

1) `True and False`
2) `not True or not False`
3) `3 > 5 or 5 > 3 and 4 != 4`
4) `(1 < 2 or 3 > 5) and (2 == 2 and 4 != 5)`
5) `not (True or False) or True and (not False)`

**A)** 0 **B)** 1 **C)** 2 **D)** 3 **E)** 4

# Decisions

***Decisions*** allow the program to perform different actions based on conditions. Python decision syntax:

```
if condition:
    statement
```

Done if condition is `True`

```
if condition:
    statement
else:
    statement
```

Done if condition is `False`

- The statement after the `if` condition is only performed if the condition is `True`.
- If there is an `else`, the statement after the else is done if condition is `False`.
- Indentation is important! Remember the colon!

# Decisions `if/elif` Syntax

If there are more than two choices, use the `if/elif/else` syntax:

```
if condition:
    statement
elif condition:
    statement
elif condition:
    statement
else:
    statement
```

```
if n == 1:
    print("one")
elif n == 2:
    print("two")
elif n == 3:
    print("three")
else:
    print("Too big!")
print("Done!")
```

# Decisions: Block Syntax

Statements executed after a decision in an `if` statement are indented for readability. This indentation is also how Python knows which statements are part of the block of statements to be executed.

- If you have more than one statement, make sure to indent them. Be consistent with either using tabs or spaces. Do not mix them!

```
if age > 19 and name > "N":
    print("Not a teenager")
    print("Name larger than N")
else:
    print("This is statement #1")
    print(" and here is statement #2!")
```

# Question: Decisions

***Question:*** What is the output of the following code?

```
n = 3
if n < 1:
    print("one")
elif n > 2:
    print("two")
elif n == 3:
    print("three")
```

**A)** nothing **B)** `one` **C)** `two` **D)** `three`

# Question: Decisions (2)

***Question:*** What is the output of the following code?

```
n = 3
if n < 1:
   print("one")
elif n > 2
   print("two")
else:
   print("three")
```

**A)** nothing

**B)** one

**C)** two

**D)** three

**E)** error

# Question: Decisions (3)

***Question:*** What is the output of the following code?

```
n = 1
if n < 1:
    print("one")
elif n > 2:
    print("two")
else:
    print("three")
print("four")
```

**A)** nothing

**B)** one
four

**C)** three

**D)** three
four

**E)** error

# Question: Decisions (4)

***Question:*** What is the output of the following code?

```
n = 0
if n < 1:
    print("one")
    print("five")
elif n == 0:
    print("zero")
else:
    print("three")
print("four")
```

**A)** nothing

**B)**
```
one
four
```

**C)**
```
one
five
four
```

**D)**
```
one
five
zero
four
```

**E)** error

# Try it: Decisions

***Question 1:*** Write a Python program that asks the user for a number then prints out if it is even or odd.

***Question 2:*** Write a Python program that asks the user for a number between 1 and 5 and prints out the word for that number (e.g. 1 is one). If the number is not in that range, print out error.

# Loops and Iteration

A ***loop*** repeats a set of statements multiple times until some condition is satisfied.

- Each time a loop is executed is called an ***iteration***.

A `for` loop repeats statements a number of times.

A `while` loop repeats statements while a condition is `True`.

# The `while` Loop

The most basic looping structure is the ***while*** loop.

A while loop continually executes a set of statements **while** a condition is true.

Syntax:

```
while condition:
    statements
```

Example:

```
n = 1
while n <= 5:
    print(n)
    n = n + 1    # Shorthand: n += 1
```

Question: What does this print?

# Question: `while` Loop

***Question:*** What is the output of the following code?

```
n = 4
while n >= 0:
    n = n - 1
    print(n)
```

**A)** numbers 3 to -1 **B)** numbers 3 to 0 **C)** numbers 4 to 0

**D)** numbers 4 to -1 **E)** numbers 4 to infinity

# Question: `while` Loop (2)

***Question:*** What is the output of the following code?

```
n = 1
while n <= 5:
    print(n)
n = n + 1
```

**A)** nothing **B)** numbers 1 to 5 **C)** numbers 1 to 6 **D)** lots of 1*s*

# The `for` Loop

A `for` loop repeats statements a given number of times.

Python `for` loop syntax:

```
for i in range(1,6):
    print(i)
```

Up to but not including ending number

Starting number

# Using `range`

The basic form of range is:

```
range(start,end)
```

- start is inclusive, end is not inclusive
- default increment is 1

May also specify an increment:

```
range(start, end, increment)
```

or just the end:

```
range(end)
```

# For Loop and While Loop

The `for` loop is like a short-hand for the `while` loop:

```
i=0
while i < 10:
    print(i)
    i += 1
```

```
for i in range(0, 10, 1):
    print(i)
```

# Common Problems – Infinite Loops

***Infinite loops*** are caused by an incorrect loop condition or not updating values within the loop so that the loop condition will eventually be false.

Example:

```
n = 1
while n <= 5:
    print(n)
    # Forgot to increase n -> infinite loop
```

# Common Problems – Off-by-one Error

The most common error is to be "***off-by-one***". This occurs when you stop the loop one iteration too early or too late.

Example:

- This loop was supposed to print 0 to 10, but it does not.

```
for i in range(0,10):
    print(i)
```

Question: How can we fix this code to print 0 to 10?

# Question: for Loop

***Question:*** How many numbers are printed with this loop?

```
for i in range(1,10):
    print(i)
```

**A)** 0 **B)** 9 **C)** 10 **D)** 11 **E)** error

# Question: for Loop

***Question:*** How many numbers are printed with this loop?

```
for i in range(11,0):
    print(i)
```

**A)** 0 **B)** 9 **C)** 10 **D)** 11 **E)** error

# Try it: `for` Loops

***Question 1:*** Write a program that prints the numbers from 1 to 10 then 10 to 1.

***Question 2:*** Write a program that prints the numbers from 1 to 100 that are divisible by 3 and 5.

***Question 3:*** Write a program that asks the user for 5 numbers and prints the maximum, sum, and average of the numbers.